RadarNet-1D: Changes to Software and Hardware and Response to the VLSID 2027 Reviews (Submission #119)

Prepared for the authors

26 October 2026

Summary

The original accelerator was re-examined before revising the paper. Its RTL did not compute the trained network correctly, and its reported latency was misread from the waveform. The hardware was therefore redesigned (`rtl_v2/`), an integer-only quantization scheme with a bit-exact golden model was introduced, and the design was verified at four levels:

1. in Python;
2. in RTL simulation (Icarus Verilog and Vivado xsim);
3. in Vivado implementation (setup and hold timing closed);
4. on the ZedBoard.

On the board it reproduces the golden model bit-exactly on 50/50 test vectors and is 60.5× faster than the same model on the on-chip ARM Cortex-A9. Every number in the revised paper is taken from these runs. Raw Vivado reports are in `results/vivado_reports/`, and the board measurement is in `results/zedboard_measured.md`.

Quantity	**Submitted paper**	**Verified now**
Cycles per inference	56	20,315 (simulation); 23,013 on board incl. AXI input
Latency	0.78 μs	142.2 μs compute; 161.9 μs measured end to end
Clock	72.2 MHz (then 100)	142.86 MHz system (met); 175.4 MHz out of context (met)
FP32 accuracy	83.23%	83.26% (`metrics.json`, reproduced in Colab)
INT8 accuracy	83.18% (weights only)	83.18% integer-only, executed bit-exactly by HW
HW correctness	not verified	50/50 bit-exact (RTL), 50/50 bit-exact (board)
LUT / FF / DSP / BRAM	10,012 / 5,461 / 0 / 15	3,117 / 2,448 / 104 / 14 (whole PL)
Power	0.671 W (vectorless)	0.290 W accelerator, 1.978 W chip incl. PS7 (vectorless)
Baseline	GPU (estimated)	ARM Cortex-A9, same chip, measured: 9.79 ms

1 Problems Found in the Submitted Design

1. **Latency.** The original `conv1d_engine` performs one MAC every two cycles, about 1.18 M MACs in total. Its own waveform (Fig. 3 of the submitted paper) shows `total_cycles = 0x244452` = 2,376,786 cycles, i.e. 32.9 ms at 72.2 MHz. The value "56 cycles" was read from the cycle counter during the LOAD state.
2. **No requantization.** Each 24-bit accumulator was saturated directly to INT8, with biases quantized at unrelated scales. The hardware therefore did not compute the trained network: in simulation it matched 0 of 4 expected classes, and the submitted Fig. 3 itself shows wrong classes (vector 0: 3 instead of 2; vector 1: 1 instead of 0).
3. **Memory timing and addressing.** The synchronous block-RAM read latency was not accounted for, so each MAC used the previous weight. The RB2 projection layer read the weights of RB2C1 from the ROM.
4. **Residual addition.** The two INT8 operands were added without aligning their scales.
5. **Accuracy claim.** The 83.18% INT8 figure came from rounding the weights only, with FP32 activations, which is not the arithmetic of the hardware.
6. **Text.** The paper claimed DSP48E1 MAC engines (Table V: 0 DSPs), a single 36 KB BRAM (15 used), and an "adaptive-precision" accelerator.

2 Software Changes

File	Change
`python/int_quant.py`	*New.* Integer-only quantization and bit-exact golden model. It folds BN, quantizes weights per channel (INT8), calibrates per-tensor activation scales (99.99th percentile, 4,096 training frames), and computes INT32 biases, fixed-point requantization multipliers $M < 2^{15}$ with per-layer shifts, and scale-aligned residual constants $M_s 2^{SK}$. Sub-commands `calib`, `eval`, `export` (ROM images and layer-descriptor table for any W), and `export-c` (C headers).
`python/preprocess.py`	Streaming loader for the 21 GB HDF5 file (about 2 GB of RAM instead of about 25 GB). Verified to produce identical arrays and therefore the identical train/test split as before.
`sw/rn_infer.c`	*New.* The same integer model in C (ARM baseline). Bit-exact with the golden model on 50/50 vectors (4-class and 7-class models).
`sw/rn_bench.c`	*New.* Bare-metal benchmark: runs every test vector on the ARM and on the accelerator over AXI, checks both against the golden logits, times both, and prints inputs, logits, predictions and a summary on the UART.
`sw/build_vitis.py`	*New.* Scripted Vitis platform and application build (Cortex-A9 / A53, `-O3`).
`colab/*.py`	*New.* One-file Colab scripts. Step 1 downloads RadioML, rebuilds the split, calibrates and evaluates the integer model. Step 2 runs the Option A pipeline.

`python/anomaly_dataset.py`
`train_anomaly.py`
`baselines.py`
`anom_eval.py`
`anom_report.py`
	New (Option A, not yet in the paper). RadioML-based jamming / spoofing / interference dataset with controlled jammer-to-signal ratio (JSR), training, an energy-detector baseline, an expert-feature classifier, and ROC / P_d-at-fixed-P_{fa} evaluation.

3 Hardware Changes

3.1 Architecture (`rtl_v2/src/`)

Original module	Replaced by
`conv1d_engine` (one per layer, serial)	One shared `rn_pe_array`: $W \times W$ INT8 multipliers (DSP48) with registered adder trees, W^2 MACs per cycle.
`proj_conv`, `fc_layer`	Ordinary layers on the same engine (1×1 stride-2 projection; FC with L=1).
Top-level 8-state FSM	Descriptor-driven sequencer in `rn_top`: loop nest $o_g \to t \to i_g \to k$, layer table generated by Python.
Skip-add in FSM, `relu`	Six-stage requantization pipeline R0–R5: bias, $\times M$, + shortcut$\cdot M_s 2^{SK}$, rounding, shift, ReLU, saturation.
8 per-layer output RAMs	Three ping-pong feature buffers (`rn_buf`), word = W channels.
`weight_rom` (BRAM IP, 8 bit)	Wide weight ROM ($W \times W$ weights per row) and parameter ROM (INT32 bias, M per channel).
`gap_unit`	Lane-parallel `rn_gap` with rounding shift.
`argmax_out` (4 classes)	Sequential argmax over any number of classes (7-class model verified).
—	`rn_axi`: AXI4-Lite wrapper (register map, debug ports for ILA/VIO); ZedBoard and Zynq UltraScale+ system scripts.

3.2 Timing closure (pipelining)

At a 5 ns target the first version reached about 105 MHz. Each round below was re-verified bit-exact before the next.

1. Two-stage argmax; reset synchronizer.
2. Registered GAP datapath.
3. An R0 register stage before the requantization multipliers.
4. Registered layer descriptors.
5. DSP48 mapping of the PE multipliers, and a separate shift stage.
6. Two-cycle BRAM reads (output registers) and two-stage address generation.
7. Pre-scaled shortcut constant, so the DSP operand comes straight from a register; equality loop-bound compares.

Result. Internal setup and hold slack are met at 169.5 / 175.4 / 138.9 MHz for $W = 4/8/16$. These frequencies are confirmed by re-implementing each design at that frequency, with the clock on a global buffer. The full ZedBoard system meets all constraints at 142.86 MHz (WNS +0.385 ns, WHS +0.053 ns, WPWS +2.520 ns).

3.3 Verification

- **RTL simulation.** All four INT32 logits match the golden model on 50/50 vectors for $W = 4$, 8, 16 and 32, for the 4-class and the 7-class model, in Icarus Verilog and in Vivado xsim.
- **AXI wrapper.** Verified with an AXI4-Lite testbench. An initial handshake-ordering bug was found and fixed.
- **Board.** ZedBoard, user-built bitstream and Vitis application: 50/50 bit-exact. Cortex-A9 9,787 μs vs. accelerator 161.9 μs end to end (60.5×).

4 Response to the Reviewers

We thank the reviewers for their careful reading and constructive comments. Re-examining the design in response to them showed that the latency reported in the submission was wrong and that the original RTL did not execute the trained network correctly. The accelerator was therefore redesigned and re-verified, and the revised manuscript reports only values obtained from simulation, Vivado implementation, and measurement on the ZedBoard. Our responses follow each comment, with references to the revised manuscript.

Reviewer 1

Comment 1.1 — Incremental novelty
BN folding, INT8 quantization, GAP through a shift, residual blocks, MAC engines, BRAM weight storage, and FSM-controlled RTL are all established techniques. The authors should clarify what hardware technique is genuinely new beyond combining these established components.

Response: We agree that these building blocks are individually well established, and we no longer present them as contributions. The contributions of the revised paper (Sec. I) are:

1. an integer-only INT8 scheme for a residual 1D CNN, with per-channel weight scales, 32-bit biases, fixed-point requantization, and *scale-aligned* residual additions (Eq. 6). Its bit-exact golden model is used for verification from Python through RTL simulation to the physical board;
2. a parameterizable $W \times W$ processing-element architecture in which a single PE array is time-multiplexed over all layers by a descriptor-driven sequencer, so the network is defined by a generated table rather than by per-layer hardware. A design-space exploration over W with timing-closed frequencies quantifies the latency–resource trade-off (Table VI);
3. on-board validation: bit-exact results on silicon, and a measured comparison against the same model on the device's own ARM processor (Table IX).

Comment 1.2 — Class detection
How are the 24 RadioML modulation classes mapped to Normal, Jammer, Spoofer, and Interference? Also, please share how modulation type can reliably identify these operational categories, particularly Jammer.

Response: The complete mapping is now given in Table I. We agree that a modulation type alone does not establish that a signal is a jammer, and the revised text states this explicitly. The grouping is presented as a candidate operational mapping used to demonstrate edge-deployed signal categorization, and its validation against operational signal libraries is stated as future work. In addition, we have built a RadioML-based dataset in which actual interference waveforms are injected into RadioML signals at controlled jammer-to-signal ratios:

- barrage, tone, swept, and pulsed jamming;
- replay spoofing;
- co-channel interference.

It is evaluated against energy-detector and expert-feature baselines. The accelerator runs this seven-class model without hardware changes; it has been verified bit-exactly in RTL simulation. This extension is outlined as future work in the Conclusion.

Comment 1.3 — Inference latency
The accelerator loads 128 I/Q pairs but claims only 56 cycles for complete inference. How are the 128 samples loaded within this latency?

Response: The reviewer is correct: the 56-cycle figure was wrong. It was read from the cycle counter during the LOAD state, and the original serial datapath in fact required about 2.38 million cycles. The redesigned accelerator performs 64 MACs per cycle. One inference takes 20,315 cycles from start to result in

RTL simulation, including the 128-cycle frame load, i.e. 142.2 μs at 142.86 MHz. On the ZedBoard, the samples are written by the ARM processor over AXI4-Lite and the cycle counter reads 23,013 cycles; the measured end-to-end latency, including this transfer, is 161.9 μs (Sec. IV-B, Table IX).

Comment 1.4 — Comparison with the state of the art
The proposed accelerator should be quantitatively compared with the prior FPGA-based RF anomaly detector by Moss et al., which also targets real-time FPGA anomaly detection.

Response: The work of Moss et al. [10] uses an unsupervised autoencoder that flags out-of-distribution signals, whereas our design is a supervised classifier that assigns operational categories. The two tasks and their metrics differ, so the comparison in the revised introduction (Sec. I) contrasts the two approaches. A quantitative entry for [10] will be added to the comparison table (Table V), reporting its published latency, throughput and resource figures alongside ours.

Reviewer 2

Comment 2.1 — Algorithmic description
There is no algorithmic description of the proposed methodology.

Response: A new subsection (Sec. III-A) describes the complete integer inference algorithm:
- BN folding (Eqs. 3–4);
- per-channel weight quantization (Eq. 5);
- activation calibration and INT32 bias computation;
- the requantization and residual-addition step executed by the hardware (Eq. 6), with the accumulator-overflow bound.

Sec. III-B describes the hardware algorithm: the loop nest (output-channel group, time step, input-channel group, kernel tap), the layer sequence, and the buffer allocation.

Comment 2.2 — Block architecture
There is no overall block/architecture of the proposed hardware.

Response: Two new figures describe the hardware:
- Fig. 2 shows the top-level datapath: sequencer, three feature buffers, weight and parameter ROMs, PE array, accumulators, requantization pipeline with residual input, GAP unit, and argmax;
- Fig. 3 details one PE lane: the DSP48 multipliers, the registered adder tree, the accumulator, and the six-stage requantization pipeline.

The integration with the Zynq processing system (AXI4-Lite register interface, clocking, reset) is described in Sec. III-B.

Comment 2.3 — Comparison with recent work
Comparisons with the recent state-of-the-art publications are missing.

Response: The comparison table (Table V) has been revised. The entries for our design now report verified values: accuracy of the integer model, compute and measured latency, accelerator power, resources, parameter storage, and on-board validation. The design-space exploration (Table VI) and the measured ARM comparison (Table IX) add design-internal and same-platform comparisons. The discussion now positions the design against streaming and binarized accelerators by operating point, rather than by latency alone.

Reviewer 3

Comment 3.1 — Novelty
The novelty of the proposed work appears to be limited, as the techniques employed are well-established and commonly used in similar FPGA-based accelerator architectures. The authors should clearly identify and substantiate the specific novel contributions.

Response: Please see our response to Comment 1.1. The contributions are restated in Sec. I, and each is substantiated by results: Table IV (integer-only accuracy), Table VI (design-space exploration with timing closure), and Table IX (bit-exact on-board measurement).

Comment 3.2 — Operating frequency
What is the rationale for selecting an operating frequency of 72.2 MHz? The authors should justify this frequency and explain whether it is determined by the FPGA implementation constraints or by a specific architectural consideration.

Response: The 72.2 MHz figure was the maximum frequency estimated from a failing 100 MHz implementation of the original design, which had long combinational paths. The redesigned datapath is pipelined throughout:
- registered multipliers and adder-tree levels;

- BRAM output registers;
- two-stage address generation;
- a six-stage requantization pipeline;
- registered control.

Out of context, the W=8 accelerator closes timing (setup and hold) at 175.4 MHz. The system clock of 142.86 MHz is the fastest Zynq PS clock step (1000/7 MHz) at which the complete system, including the AXI interconnect, meets timing with margin: WNS +0.385 ns, WHS +0.053 ns (Sec. IV-A).

Comment 3.3 — LOAD and inference cycles
The paper reports that the LOAD operation and inference can be completed within 56 clock cycles. This appears to be very aggressive and requires further clarification.

Response: Please see our response to Comment 1.3. The corrected figures are 20,315 cycles in simulation and 161.9 μs measured on the board, both including the input load.

Comment 3.4 — FPGA-SoC block diagram
A detailed FPGA-SoC block diagram is not provided. Including a detailed block diagram showing the processor, accelerator, memory hierarchy, interconnects, and other major components, along with relevant implementation details, would provide better clarity on the overall system architecture and implementation.

Response: A system block diagram has been prepared. It shows:
- the ARM Cortex-A9 processing system, DDR and on-chip memory;
- the AXI interconnect and the AXI4-Lite accelerator interface with its register map;
- the accelerator, the clock and reset generation, and the JTAG host connection.

The accelerator's internal memory hierarchy (three feature buffers, weight and parameter ROMs) is shown in Fig. 2. Implementation details of the system are given in Sec. III-B and Tables VII–VIII.

Reviewer 4

Comment 4.1 — Taxonomy
The four-class operational taxonomy (Normal/Jammer/Spoofer/Interference) and its mapping from RadioML 2018.01A's 24 classes is not clearly described.

Response: Please see our response to Comment 1.2. The mapping is listed in Table I, together with the preprocessing used to build the balanced 30,000-sample test set (Sec. II).

Comment 4.2 — Accuracy
83.23% accuracy is low for the stated mission. For a security-critical task (countering jammers/spoofers in microseconds), a ∼17% error rate is substantial.

Response: We agree. The per-class results (Table III) show where the errors come from:
- Normal and Spoofer are recognized with F1 scores of 99% and 95%;
- nearly all errors are confusions between Jammer and Interference, because higher-order QAM modulations are split between these two groups and cannot be reliably distinguished from 128 samples.

The limitation therefore lies in the class definition rather than in the hardware: the accelerator reproduces the model bit-exactly, at 83.18% compared with 83.26% in FP32. This is why we are moving to the injected-interference dataset described in our response to Comment 1.2. In that dataset the classes correspond to physically distinct interference types.

Comment 4.3 — Baseline fairness
The GPU baseline comparison is methodologically unfair — comparing FPGA datapath latency against batch-1 GPU end-to-end latency inflates the claimed advantage; fairer baselines (on-chip ARM cores, MCU-class processors, or comparable FPGA accelerators) should be used.

Response: We agree, and the GPU comparison has been removed. The revised paper uses the reviewer's suggested baseline. The same integer model, implemented in C and compiled with `-O3`, runs on the ARM Cortex-A9 of the same Zynq device, and both implementations are measured on the board with the same timer. The ARM takes 9.79 ms per inference; the accelerator takes 161.9 μs end to end, including data transfer. Both produce logits bit-identical to the golden model (Table IX). Comparable FPGA accelerators are contrasted in Table V.

Comment 4.4 — Clock frequency
The 72.2 MHz clock frequency is unusually low for the Zynq-7000 platform, suggesting potential timing closure issues or a serialized datapath with limited parallelism.

Response: The reviewer's diagnosis was correct: the original datapath was both serialized and not pipelined. Please see our response to Comment 3.2. The redesign performs W^2=64 MACs per cycle and runs at 142.86 MHz in the complete system (175.4 MHz out of context).